



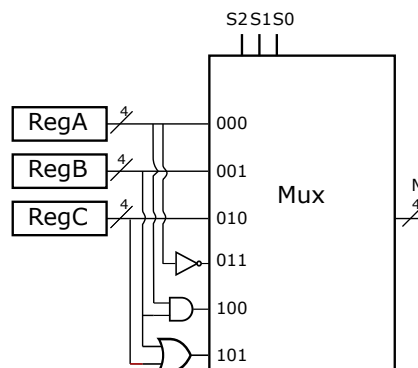
Duración: 120 minutos

Instrucciones generales

- Puntaje: 60 pts.
- El desarrollo es estrictamente individual.
- No está autorizado el uso de dispositivos electrónicos de ningún tipo.
- **Responder CADA PREGUNTA EN UNA HOJA SEPARADA. Se descontará 0.1 por no seguir esto.**
- La notación 0x indica que los dígitos que siguen son hexadecimales. La notación 0b indica que los dígitos que siguen son binarios. Ambas notaciones no son parte del número.
- Para cada ejercicio se recomienda explicitar el procedimiento utilizado. En caso que el resultado esté incorrecto pero haya procedimiento, podría obtener puntaje parcial.

1. [16p] PREGUNTA 1. Multiplexores y demultiplexores

- 1.1) [4p] Considere el multiplexor de la izquierda con 3 bit de selección y 6 entradas de 4 bit. Indique la secuencia de bits de selección para que la salida M del multiplexor sea (a) RegB, (b) RegC, (c) NOT RegA (d) RegB OR RegC.



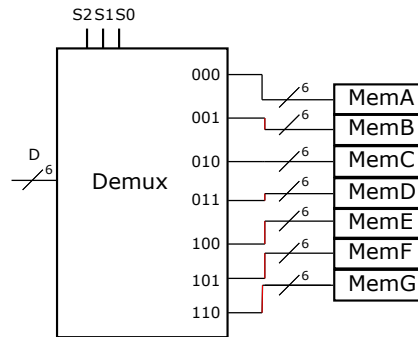
R.

	S2	S1	S0	M
(a)	0	0	1	RegB
(b)	0	1	0	RegC
(c)	0	1	1	NOT RegA
(d)	1	0	1	RegB OR RegC

1p por cada grupo S2, S1, S0 correcto

0p por cada grupo S2, S1, S0 incorrecto

- 1.2) **[4p]** Considere el demultiplexor de la derecha con 3 bit de selección y 7 salidas de 6 bit. Indique la secuencia de bit de selección para que el valor de entrada D se almacene en (a) MemE, (b) MemG, (c) MemB, (d) MemD



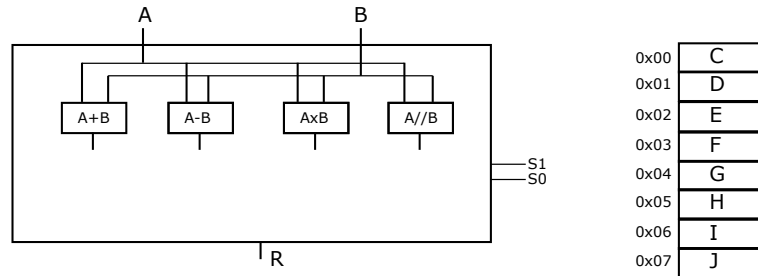
R.

	S2	S1	S0	Destino
(a)	1	0	0	MemE
(b)	1	1	0	MemG
(c)	0	0	1	MemB
(d)	0	1	1	MemD

1p por cada grupo S2, S1, S0 correcto

0p por cada grupo S2, S1, S0 incorrecto

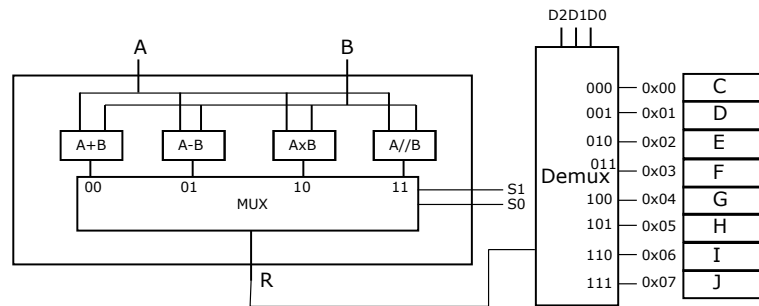
- 1.3) [8p] Considere una ALU incompleta que calcula 4 operaciones: suma, resta, producto y división entera. Este circuito recibe dos entradas A y B de 4 bit cada una, y posee 2 bit selectores. La salida R está conectada a una memoria de 8 posiciones de 4 bit cada una.



- a) [4p] Complete el funcionamiento de este sistema usando los componentes que necesite (incluyendo multiplexores y demultiplexores).

R.

Puede haber varias maneras de hacerlo, pero la más directa es tener un multiplexor dentro de la ALU y agregar un demultiplexor entre R y la memoria.



2p. Agregar el multiplexor y conectarlo dentro de la ALU. Debe darle un nombre a los selectores del multiplexor y del demultiplexor y conectar las entradas y salidas.

1p. Error menor pero en la dirección correcta. Puede faltar alguna conexión.

0p. Incorrecto.

Si hay soluciones alternativas, evaluar si cumplen con lo pedido. 2p si cumple. 1p si cumple con algún error menor, pero la idea general está bien. 0p si no cumple.

- b) [4p] Indique los valores de los bit de selección para realizar las siguiente acciones: (a) $C = A + B$, (b) $F = A - B$, (c) $H = A + B$, (d) $J = A - B$.

R.

Aquí deben ser consecuentes con los bit de selección que definieron en la pregunta anterior. Pueden estar en cualquier orden, pero deben ser consistentes con su diagrama.

	S1	S0	D2	D1	D0	Operación
(a)	0	0	0	0	0	$C = A + B$
(b)	0	1	0	1	1	$F = A - B$
(c)	0	0	1	0	1	$H = A + B$
(d)	0	1	1	1	1	$J = A - B$

2p. Por cada grupo de selectores correcto.

1p. Si hay algún error menor, pero va en la dirección correcta.

0p. Incorrecto.

2. [28p] PREGUNTA 2. Ejecución de instrucciones

2.1) [18p] A partir del esquema de computador básico mostrado en clases. Suponga que los registros RegA y RegB se inicializan con los valores 0x04 y 0x05 respectivamente.

La memoria de instrucciones contiene los siguientes valores.

dirección (8 bit)	contenido (14 bit)
0x00	0x04 0x00
0x01	0x0E 0x00
0x02	0x18 0xFF
0x03	0x08 0x35
0x04	0x03 0x38
0x05	0x19 0x4B
0x06	0x07 0x07

Complete la siguiente tabla indicando los valores de iniciales y finales de los registros en cada paso, el valor que sale por cada multiplexor, y la instrucción *assembler* ejecutada en cada caso.

RegA inicial	RegB inicial	PC	Instrucción	MUX A	MUX B	RegA final	RegB final
0x04	0x05	0x00	ADDA				

R.

RegA inicial	RegB inicial	PC	Instrucción	MUX A	MUX B	RegA final	RegB final
0x04	0x05	0x00	ADDA	0x04	0x05	0x09	0x05
0x09	0x05	0x01	ORB	0x09	0x05	0x09	0x0D
0x09	0x0D	0x02	SHLL 0xFF	0x09	0xFF	0xFE	0x0D
0xFE	0x0D	0x03	SUBB	0xFE	0x0D	0xFE	0xF1
0xFE	0xF1	0x04	MOVBL 0x38	0x00	0x38	0xFE	0x38
0xFE	0x38	0x05	SHRA	0xFE	0x38	0x7F	0x38
0x7F	0x38	0x06	SUBA	0x7F	0x38	0x47	0x38

No es necesario especificar los cálculos.

2p por incrementar los PCs de manera correcta.

2p por traducir las instrucciones a assembler de manera correcta

Adicionalmente, 2p para cada fila (7 filas): 2p si están todos los valores correctos; 1p si hay un error menor; 0p si está incorrecto. Si alguna fila tiene un error, descontar la primera vez y seguir revisando como si la fila anterior estuviera correcta. La idea es no castigar el mismo error más de una vez (error de arrastre).

- 2.2) [6p] Utilizando el mismo esquema de computador básico, escriba una secuencia de instrucciones (un programa) que cargue el valor 0x1F en RegA, el valor 0xC6 en RegB, y a continuación deje en RegB el doble del resultado del AND entre ambos valores.

Para escribir la secuencia de instrucciones, muestre el contenido que tendría la memoria de instrucciones indicando la dirección y el contenido, de manera similar a la pregunta anterior. Utilice los nombres de las instrucciones en *assembler* en lugar del OPCODE.

Nota: **No es válido cargar directamente el resultado final en RegB.** Debe calcularlo utilizando las instrucciones disponible en el computador.

R.

Puede haber varias maneras de hacerla. Una es:

dirección (8 bit)	contenido (14 bit)
0x00	MOVAL 0x1F, almacena 0x1F en RegA
0x01	MOVBL 0xC6, almacena 0xC6 en RegB
0x02	ANDB, calcula $A \wedge B$ y lo deja en RegB
0x03	SHLB, multiplica por dos el contenido de RegB

Otra alternativa es hacer el ANDA, multiplicar con SHLA y luego copiar a B con MOVBA.

dirección (8 bit)	contenido (14 bit)
0x00	MOVAL 0x1F, almacena 0x1F en RegA
0x01	MOVBL 0xC6, almacena 0xC6 en RegB
0x02	ANDA, calcula $A \wedge B$ y lo deja en RegA
0x03	SHLA, multiplica por dos el contenido de RegA
0x04	MOVBA, copia el valor de A a B

Para multiplicar por 2 el valor de B pueden copiar B a A, y sumar ambos registros en B. Esta versión hace eso.

dirección (8 bit)	contenido (14 bit)
0x00	MOVAL 0x1F, almacena 0x1F en RegA
0x01	MOVBL 0xC6, almacena 0xC6 en RegB
0x02	ANDB, calcula $A \wedge B$ y lo deja en RegB
0x03	MOVAB, copia el valor de B a A
0x04	ADDB, suma ambos valores y deja el resultado en B

En todos estas versiones no se necesario agregar una explicación de lo que hace cada instrucción. Aquí está solo para efectos de claridad.

Cualquier solución de estas (o alguna mezcla) sirve mientras cumpla los requisitos de: (1) cargar los valores correctos en RegA y en RegB, (2) calcular el AND ya sea en RegA o en RegB, (3) multiplicar por dos el resultado, (4) dejar el resultado en RegB.

1p. Carga los valores correctos en ambos registros.

2p. Calcular el AND.

2p. Multiplicar por dos.

1p. Dejar el resultado final en RegB

- 2.3) [4p] Proponga un OPCODE nuevo, que calcule la operación $B = \neg B$. Indique la línea completa que debería agregarse al set de instrucciones para implementar este nuevo OPCODE (puede inventar el nombre de la instrucción).

R.

La nueva instrucción, que en este caso llamaremos BNOTB necesita indicar apropiadamente los valores de $L_a = 0$ y $L_b = 0$ para que el resultado quede en B . Los MUX deben estar configurados para que MUXA **no** selecciona el valor de RegA, y para que MUX seleccione el valor de RegB. El valor de Sop puede ser 100, que es el valor que permite que la ALU calcule $\neg A$ ¹.

Instrucción	OPCODE	La	Lb	Ma0	Mb1	Mb0	Sop2	Sop1	Sop0	Operación (significado)
BNOTB	111111	0	1	1	0	0	1	0	0	$B = \neg B$

1p. Elegir un OPCODE **no repetido** y un nombre de instrucción cualquier (**no repetido**).

1p. Configurar correctamente L_a y L_b .

1p. Configurar correctamente M_{a0} , M_{b1} y M_{b0} .

1p. Elegir el Sop 100. O bien justificar por qué no podría ser ése (u otro).

¹Técnicamente el hecho que la ALU calcule $\neg A$ con Sop= 100 no implica que al entrar 0 por RegA y otro valor por RegB, esto vaya a calcular $\neg B$. Sin embargo, para efectos de la pregunta sí es aceptable. También es aceptable si discuten este problema.

3. **[16p] PREGUNTA 3.** Sistemas Operativos

3.1) **[3p]** Indique los 3 componentes principales de un computador de acuerdo a la arquitectura mencionada en clases.

R.

- CPU
- Memoria
- Dispositivos de entrada/salida (o input/output). También está bien si dicen “disco”.

1p por cada componente.

3.2) **[2p]** ¿Qué significa que un procesador pueda funcionar a 1GHz (1Giga = 10^9) en términos de instrucciones procesadas?

R.

Que funcione 1GHz implica que puede procesar $10^9 = 1000000000$ instrucciones por segundo. Pueden argumentar también que es “a lo más” 10^9 instrucciones por segundo.

2p. Correcto.

1p. Error menor.

0p. Incorrecto o vacío.

- 3.3) [3p] Suponga que tenemos un procesador con un núcleo que procesa $1M = 10^6$ instrucciones por segundo. Si ahora el procesador posee 4 núcleos, ¿es verdad que los programas ahora funcionarán 4 veces más rápido? Justifique la respuesta.

R.

No es necesariamente cierto. De hecho, en la mayoría de los casos no lo será. Depende de cuán paralelizable son las instrucciones del programa o si hay dependencias entre ellas (una debe ser necesariamente ejecutada antes que otra). Por ejemplo, las instrucciones:

```
a = 4
b = 8
c = 4 + 8
d = 2 + 4
```

pueden ser ejecutadas, cada una en un núcleo distinto y solo tomaría el mismo tiempo que una instrucción (técnicamente las sumas podrían tomar un ciclo más). El programa se acelera casi 4 veces. Por otro lado, si las instrucciones son:

```
a = 4
b = a + 4
c = a + b
d = 2 + c
```

necesariamente deben ser ejecutadas en el orden indicado, una después de la otra. Por lo tanto aquí el programa no tendría ninguna ventaja al ser ejecutado en un sistema con cuatro núcleos.

1p. Indicar que no.

2p. Justificación de la respuesta. No es necesario que hagan un contraejemplo, pero tiene que haber una justificación razonable. (si dijeron que sí, tendría que haber una justificación acorde, pero esta no puede tener más de 1p).

- 3.4) **[6p]** ¿Cuál(es) de las siguientes acciones son tareas que debe realizar un sistema operativo, versus una tarea que realiza una aplicación? (a) Compilar un programa, (b) Ejecutar un programa, (c) Permitir a un programa leer y escribir en el disco, (d) Editar un programa, (e) Desenscriptar un documento, (f) Entrenar un modelo de inteligencia artificial.

R.

Tareas propias del sistema operativo:

- (b) Ejecutar un programa. Implica asignar memoria, modificar Program Counter y registros.
- (c) Permitir a una programa leer y escribir en el disco. Implica modificar algo por *hardware*.

Tareas propias de una aplicación:

- (a) Compilar un programa. Es una transformación de un archivo a otro.
- (d) Editar un programa. Corresponde a modificar parte del programa.
- (e) Desenscriptar un documento. También corresponde a transformar una secuencia de bytes de un formato a otro.
- (f) Entrenar un modelo de inteligencia artificial. Implicar ejecutar muchos cálculos pero no modificar elementos del sistema operativo.

1p. Por cada afirmación correcta/incorrecta. No se necesita justificación.

3.5) [2p] ¿Cuál es el rol de las llamadas al sistema (*syscalls*) en un sistema operativo?

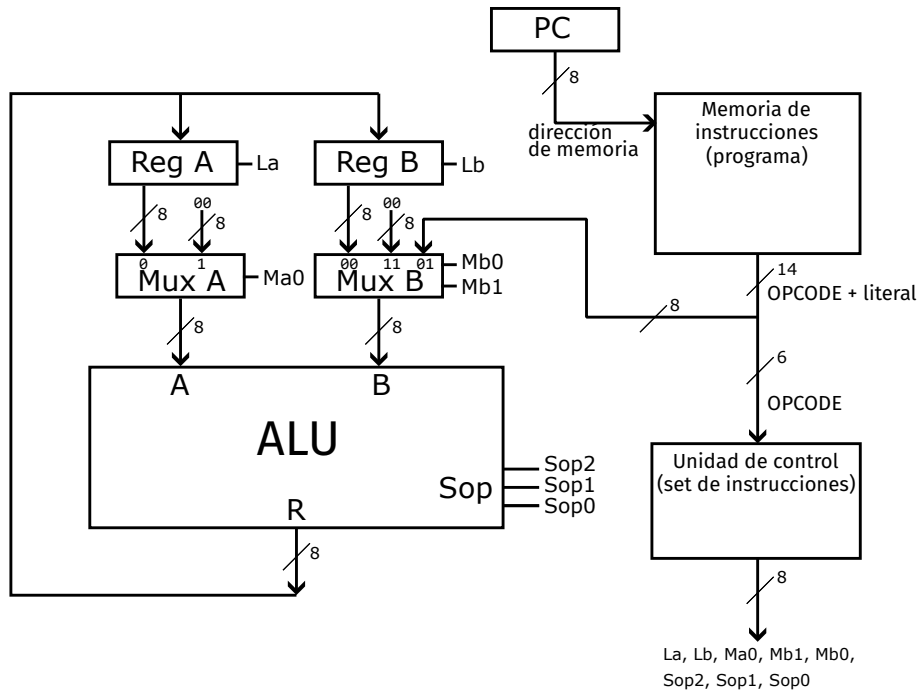
R.

Las llamadas al sistema permiten que una aplicación pueda solicitar servicios o ejecutar acciones que son propias del sistema operativo. Entre ellas están: abrir un archivo, leer/escribir del disco, enviar mensaje por la red, crear procesos nuevos, asignar memoria a un programa, y otras.

2p. Respuesta correcta. Indica que solicita acciones al sistema operativo.

1p. Error menor pero en la dirección correcta.

0p. Incorrecto o vacío.



Instrucción	OPCODE	Set de instrucciones								Operación (significado)
		La	Lb	Ma0	Mb1	Mb0	Sop2	Sop1	Sop0	
MOVAB	000000	1	0	1	0	0	0	0	0	$A = B$
MOVBA	000001	0	1	0	1	1	0	0	0	$B = A$
MOVAL	000010	1	0	1	0	1	0	0	0	$A = \text{Literal}$
MOVBL	000011	0	1	1	0	1	0	0	0	$B = \text{Literal}$
ADDA	000100	1	0	0	0	0	0	0	0	$A = A + B$
ADDB	000101	0	1	0	0	0	0	0	0	$B = A + B$
ADDL	000110	1	0	0	0	1	0	0	0	$A = A + \text{Literal}$
SUBA	000111	1	0	0	0	0	0	0	1	$A = A - B$
SUBB	001000	0	1	0	0	0	0	0	1	$B = A - B$
SUBL	001001	1	0	0	0	1	0	0	1	$A = A - \text{Literal}$
ANDA	001010	1	0	0	0	0	0	1	0	$A = A \wedge B$
ANDB	001011	0	1	0	0	0	0	1	0	$B = A \wedge B$
ANDL	001100	1	0	0	0	1	0	1	0	$A = A \wedge \text{Literal}$
ORA	001101	1	0	0	0	0	0	1	1	$A = A \vee B$
ORB	001110	0	1	0	0	0	0	1	1	$B = A \vee B$
ORL	001111	1	0	0	0	1	0	1	1	$A = A \vee \text{Literal}$
NOTA	010000	1	0	0	0	0	1	0	0	$A = \neg A$
NOTB	010001	0	1	0	0	0	1	0	0	$B = \neg A$
NOTL	010010	1	0	0	0	1	1	0	0	$A = \neg \text{Literal}$
XORA	010011	1	0	0	0	0	1	0	1	$A = A \oplus B$
XORB	010100	0	1	0	0	0	1	0	1	$B = A \oplus B$
XORL	010101	1	0	0	0	1	1	0	1	$A = A \oplus \text{Literal}$
SHLA	010110	1	0	0	0	0	1	1	0	$A = \text{Shift left}(A)$
SHLB	010111	0	1	0	0	0	1	1	0	$B = \text{Shift left}(A)$
SHLL	011000	1	0	0	0	1	1	1	0	$A = \text{Shift left}(\text{Literal})$
SHRA	011001	1	0	0	0	0	1	1	1	$A = \text{Shift right}(A)$
SHRB	011010	0	1	0	0	0	1	1	1	$B = \text{Shift right}(A)$
SHRL	011011	1	0	0	0	1	1	1	1	$A = \text{Shift right}(\text{Literal})$